



湖北大学
Hubei University

计算机科学与工程学院

第 22 届湖南省大学生计算机程序设计竞赛
-----应用开发类竞赛（2026）

系 统 设 计 说 明 书

作品名称：码安管家——LLM 驱动的漏洞分析站

作品类别：信息安全类

作 者：白明耀、易卓玥、谭依晴

指导老师：陈纪友

单 位：衡阳师范学院

二零二六年九月

目 录

1 项目概述.....	1
1.1 作品名称与定位.....	1
1.2 开发背景.....	1
1.3 目标用户与使用场景.....	2
1.4 术语与缩略语.....	2
2 功能需求说明书.....	3
2.1 功能总览.....	3
2.2 核心扫描任务.....	4
2.2.1 任务 A：扫描代码片段或单个文件.....	4
2.2.2 任务 B：批量扫描项目并导出报告.....	5
2.2.3 任务 C：扫描 GitHub 仓库或网页.....	5
2.3 扫描结果与判定语义.....	5
2.4 编辑器插件功能.....	6
2.5 命令行功能.....	6
2.6 模型管理与离线能力.....	6
2.7 非功能需求.....	7
3 概要设计说明书.....	7
3.1 总体架构.....	7
3.2 模块划分.....	8
3.3 扫描引擎主线.....	9
3.4 接口设计.....	9
3.5 部署形态与运行环境.....	10

3.6 效果指标.....	11
4 详细设计说明书.....	12
4.1 Stage 1 工具召回设计.....	12
4.2 Stage 2 封闭裁决设计.....	13
4.3 2.5 代信任层.....	14
4.4 复核兜底三出口.....	15
4.5 关键工程机制.....	16
4.6 设计依据与选型论证.....	16
5 数据存储设计.....	17
5.1 总体说明.....	17
5.2 文件存储.....	17
5.3 向量知识库.....	18
5.4 前端持久化.....	18
5.5 数据生命周期.....	18
6 软件界面设计书.....	19
6.1 设计系统.....	19
6.2 信息架构与页面清单.....	19
6.3 界面总览.....	19
6.4 关键交互.....	23
7 用户操作手册.....	24
7.1 三个最常用的任务.....	24
7.2 读懂扫描结果.....	24
7.3 Web 界面导览.....	24
7.4 在编辑器里使用.....	24

7.5 命令行使用.....	25
7.6 离线使用与模型管理.....	25
7.7 常见问题与处理.....	25
8 安装手册.....	26
8.1 环境要求.....	26
8.2 获取与一键启动.....	26
8.3 首次启动流程.....	26
8.4 确认安装成功.....	27
8.5 卸载.....	27
附录.....	27
附录 A 命令行速查.....	27
附录 B 输入限制与支持范围.....	28
附录 C 排查速查.....	28

注：本目录由域代码生成。编辑后如需更新页码，请在目录上右键选择「更新域」。

1 项目概述

1.1 作品名称与定位

本作品名称为「码安管家——LLM 驱动的漏洞分析站」，参赛类别为信息安全类。软件发布名「凿凿（ZaoZao）」，取「言之凿凿」之意——每一个漏洞结论都必须附带可核对的证据链；其内核引擎代号为 Nivis（正文所称引擎、模型系列名均沿用此代号）。

凿凿是一套完全运行在用户本机电脑上的 AI 代码漏洞扫描系统。它的基本工作方式是：传统安全工具（Semgrep、Bandit、Gitleaks、Trivy 等）负责快速召回可疑点，本地部署的大语言模型再对每一个可疑点做语义裁决——它是不是真实漏洞、属于哪一类 CWE、如何修复。相比规则型工具，它能看懂数据流、识别有效防御、解释漏洞成因；相比云端智能服务，它的代码、模型与知识库全部留在本机，运行期可完全离线，代码不出门。

1.2 开发背景

软件安全漏洞造成的损失持续扩大。IBM《2024 年数据泄露成本报告》显示，全球单次数据泄露事件平均损失达 488 万美元，较上一年增长 10%；中国国家互联网应急中心（CNCERT）发布的《2024 年上半年我国互联网安全态势报告》指出，上半年收录的通用软硬件漏洞数量同比增长 17.9%，Web 应用层漏洞仍是攻击者的主要入口。代码安全审计在软件开发生命周期中的位置不断前移，成为 DevSecOps 实践的关键环节。

工业界长期依赖基于规则与数据流的静态应用安全测试（SAST）工具，如 Bandit、Semgrep、CodeQL 等。这类工具对已知漏洞模式检出快、误报可控，但规则匹配的边界在面对复杂业务逻辑、绕过式过滤与非典型漏洞变体时迅速收紧。一个典型例子：对使用 `os.path.join` 拼接用户输入的路径穿越样本，规则工具因命中「标准库推荐用法」而放行，而具备语义理解能力的大语言模型能够识别该写法不构成安全边界、攻击者可通过「`../`」实现目录穿越。这类「看似标准写法实则可绕过」的场景，难以通过扩充规则集根本解决。

大语言模型的兴起为代码安全审计提供了新路径，但闭源模型按 token 计费、数据需上传云端，与代码作为核心资产对保密性与可审计性的要求冲突；直接本地

部署通用开源代码大模型，又面临三类瓶颈：一是漏洞模式知识盲区，二是判定边界过度敏感、对安全样本产生幻觉式误报，三是 CWE 分类归因能力薄弱。本作品要回答的问题是：如何在不依赖任何云端 API 的前提下，通过两阶段架构、数据工程与监督微调，把一个 8B 参数的本地开源模型变成一套判定可信、误报可控、结论可解释的可用漏洞分析系统。

1.3 目标用户与使用场景

本系统面向需要在本地完成代码安全开发的开发者与安全审计人员，覆盖四类典型场景：

表 1-1 目标用户与使用场景

场景	使用方式
交代码前自查一段拿不准的逻辑	Web 界面粘贴代码，秒级获得三态判定与修复建议
提交/发布前把整个项目过一遍	批量扫描整个模块，导出 Markdown 报告与 SARIF 结果
评估一个开源仓库能否引入	输入仓库地址浅克隆扫描，不本机留副本
写代码时随手查当前文件	VS Code / IntelliJ 插件，编辑器内右键即扫

交付形态包括：Web 界面（五页面）、VS Code 与 IntelliJ 双编辑器插件、命令行工具，以及跨平台一键启动器与卸载器。

1.4 术语与缩略语

表 1-2 术语与缩略语

术语	说明
CWE	通用缺陷枚举（Common Weakness Enumeration），漏洞类型的国际标准编号体系，如 CWE-89 为 SQL 注入
SAST	静态应用安全测试（Static Application Security Testing），基于规则与数据流扫描源代码
污点分析	追踪外部输入（Source）到危险函数（Sink）的数据流，判断不可控数据是否到达危险点

术语	说明
LoRA / QLoRA	低秩适配微调方法；QLoRA 将基座 4-bit 量化后叠加 LoRA，使 8B 模型可在 16GB 显存训练
SFT	监督微调（Supervised Fine-Tuning），用带标注的指令数据对预训练模型做监督训练
两阶段架构	本系统核心架构：Stage 1 工具召回候选，Stage 2 大模型对候选做封闭裁决
自一致率	同一问题独立采样 N 次的票数一致比例，作为置信度代理指标
共形预测	一种统计不确定性量化方法，给出带覆盖率保证的预测集合，用于门控低置信判定
反事实验证	对 Sink 行注入防御代码后重判，观察结论是否翻转，以区分真理解与模式匹配
SARIF	静态分析结果交换格式（OASIS 标准，2.1.0），可与 GitHub Code Scanning 等互通
NDJSON	换行分隔 JSON（Newline-Delimited JSON），本系统用于批量扫描与下载进度的流式推送
FPR / 召回率	误报率（把安全判成漏洞的比例）/ 漏洞检出比例；strict 口径另要求 CWE 归因正确
RAG	检索增强生成（Retrieval-Augmented Generation），检索本地漏洞知识库辅助判定
Ollama	本地大模型运行时，支持从模型仓库在线拉取量化模型

2 功能需求说明书

2.1 功能总览

系统功能按入口分为 Web 端、编辑器插件、命令行与系统工具四组，功能清单如表 2-1 所示。

表 2-1 功能清单

入口	功能	说明
Web 端	代码扫描	粘贴代码片段或上传文件（单文件/批量最多 200 个），三态判定输出
Web 端	GitHub 仓库扫描	浅克隆远程仓库逐文件分析，扫完即删本机副本
Web 端	网页代码扫描	抓取指定网页中的脚本代码逐个分析
Web 端	结果与报告	漏洞卡片、安全评分、Markdown 报告与 SARIF 导出、报告历史
Web 端	CWE 样本库	16 类常见漏洞的可搜索示例与修复方案
Web 端	安全态势	历史扫描统计、引擎健康状态与整体安全态势总览
Web 端	模型管理	模型拉取/删除/切换，实时下载进度，后端形态展示
VS Code 插件	编辑器内扫描	右键分析当前文件，红波浪线行级标记，信任层明细面板，工作区批量扫描，保存自动扫
IntelliJ 插件	编辑器内扫描	选中代码或整文件扫描，气球通知展示结论与信任层摘要（演示版）
命令行	扫描与集成	health / scan / batch / url / github 五个子命令，可接入 CI 脚本
系统工具	一键启动/卸载	跨平台启动器自动装依赖、选后端、下模型；卸载器分级清理

2.2 核心扫描任务

2.2.1 任务 A：扫描代码片段或单个文件

用户在扫描工作台「粘贴代码」标签选择语言、粘贴代码后点击「开始分析」，或拖入一个代码文件。单文件典型耗时 5~15 秒（长文件自动切片后逐段分析，耗时

更长)。结果区给出：是否存在漏洞、CWE 类型、风险等级、漏洞原理说明与行号锚定的修复建议。

2.2.2 任务 B：批量扫描项目并导出报告

一次拖入整个模块的代码文件（最多 200 个、总计 10MB），页面通过流式接口实时显示进度，扫完一个出一个。完成后可下载 Markdown 报告（漏洞汇总表 + 逐文件详细结果 + 修复建议），亦可导出 SARIF 2.1.0 标准格式对接代码托管平台。

2.2.3 任务 C：扫描 GitHub 仓库或网页

输入仓库地址即可扫描：系统以浅克隆方式只读拉取（120 秒超时、文件数上限默认 20 可调、硬上限 50），扫描完成后立即删除本机仓库副本。「URL」标签输入网页地址，系统抓取页面中的脚本代码逐个分析。

2.3 扫描结果与判定语义

系统的每一份文件判定为三种状态之一，语义如表 2-2 所示。

表 2-2 三态判定语义

判定	含义	建议动作
发现漏洞	模型多次采样一致且通过证据校验，系统确信存在漏洞	按修复建议处理
安全	工具未召回可疑点，且全文件兜底复核亦未发现问题	常规流程即可
需人工复核	系统有怀疑但证据不足，有意不代替用户下结论	花一分钟查看标注位置

「需人工复核」是设计行为而非故障：系统的原则是宁可多一道人工复核，也不漏报真漏洞、不错杀不确定的判定。复核结果单独列出，不计入安全评分。每条结果包含：CWE 编号与名称、风险等级（Critical/High/Medium/Low）、Source 与 Sink 污点行号（有数据流的漏洞）、自然语言漏洞说明、行号锚定的局部修复建议，以及可折叠查看的模型原始输出。

安全评分采用扣分制：Critical 每项 -8、High -8、Medium -4、Low -2、Info -0.5；需人工复核的样本单列不扣分，保证评分语义与复核信号不混淆。

本系统结果与传统规则工具存在三类正常差异：一是识别有效防御后放行（参数化查询、subprocess 列表参数、shlex.quote 等），降低规则工具的常见误报；二是

报出规则工具覆盖不到的语义类漏洞（认证缺失、信任边界绕过等）；三是对低置信判定输出「需人工复核」而非强行二选一。

2.4 编辑器插件功能

VS Code 插件（v1.3.0）通过 VSIX 安装，提供：右键「分析当前文件」；结果面板除判定与修复建议外，展示每条候选的信任层明细（置信度、裁决档位、共形预测、反事实验证、证据门五项信号）；确认漏洞按行号标注红色波浪线（可配置关闭）；命令面板批量扫描工作区；保存后自动扫描（1.5 秒防抖）。后端地址、批量文件上限等均可配置。

IntelliJ IDEA 插件（v0.1.0）为演示版：编辑器选中代码（不选则扫整个文件）后右键扫描或用快捷键，结果以气球通知展示结论、修复建议与信任层摘要。两插件均为后端服务的「遥控器」，需后端服务保持运行。

2.5 命令行功能

命令行工具无需启动后端服务即可复用扫描引擎，提供五个子命令：**health**（健康检查）、**scan**（扫单文件，可输出 JSON）、**batch**（扫目录并导出报告，支持限量与不递归）、**url**（扫网页脚本）、**github**（浅克隆扫仓库）。通用参数支持切换模型、开启知识库、选择输出格式。退出码契约明确：扫描失败返回 1、中断返回 130，便于接入 CI 做提交前检查。

2.6 模型管理与离线能力

Web 界面右上角的模型管理抽屉支持模型的拉取、删除与切换，下载带实时进度；亦可通过命令行参数或环境变量切换。模型提供两档精度形态：一是发布形态 v9max（Ollama 量化版，约 4.7GB，兼容性最好，首次启动自动拉取）；二是高精度形态（Transformers 进程内叠加 LoRA 适配器，检出质量更高），启动器检测到本机自带模型且硬件可运行时自动启用，跑不动则自动回退并打印说明。界面如实显示当前使用的形态。

系统运行期完全离线：联网仅在首次下载依赖与模型时需要；推理框架强制本地离线模式，Semgrep 规则已随仓库本地化。国内网络环境下依赖走清华镜像、模型走国内镜像，支持断点续传与断流自动换源，一般无需手动配置代理。

2.7 非功能需求

表 2-3 非功能需求与约束

项目	要求
支持语言	Python / JavaScript / TypeScript / Java / PHP / Go / HTML
文件类型	.py .js .ts .jsx .tsx .java .php .go .html .htm .vue .svelte
输入限额	单段代码 200 万字符；单文件 2MB；单次批量 200 个文件且总计 10MB；超限前后端双重拦截（HTTP 413）
仓库扫描	浅克隆 + 120 秒超时 + 文件数上限（默认 20，硬上限 50）
性能预期	单文件典型 5~15 秒；长文件切片后可达分钟级；无显卡自动回退 CPU（慢 10~20 倍）
隐私安全	代码、模型、知识库全部留在本机，运行期不访问外网；仓库扫描用后即删
已知边界	跨文件复杂攻击路径与「缺失型」漏洞（认证缺失、CSRF 等）为弱项，置信不足时转人工复核；结论是审计辅助而非最终结论

3 概要设计说明书

3.1 总体架构

系统采用「前端多页面 + FastAPI 服务层 + 两阶段扫描引擎 + 多后端模型抽象 + 本地持久化」的五层结构。用户经 Web 页面、编辑器插件或命令行发起扫描请求；服务层完成校验、排队与调度后调用两阶段引擎：Stage 1 由传统安全工具与自研静态分析并行召回候选漏洞，Stage 2 由本地大模型对候选做封闭裁决，再经信任层门控与兜底复核后聚合为三态结论；结果以 JSON/NDJSON 返回前端，并可导出 Markdown 与 SARIF。整体数据流如图 3-1 所示。

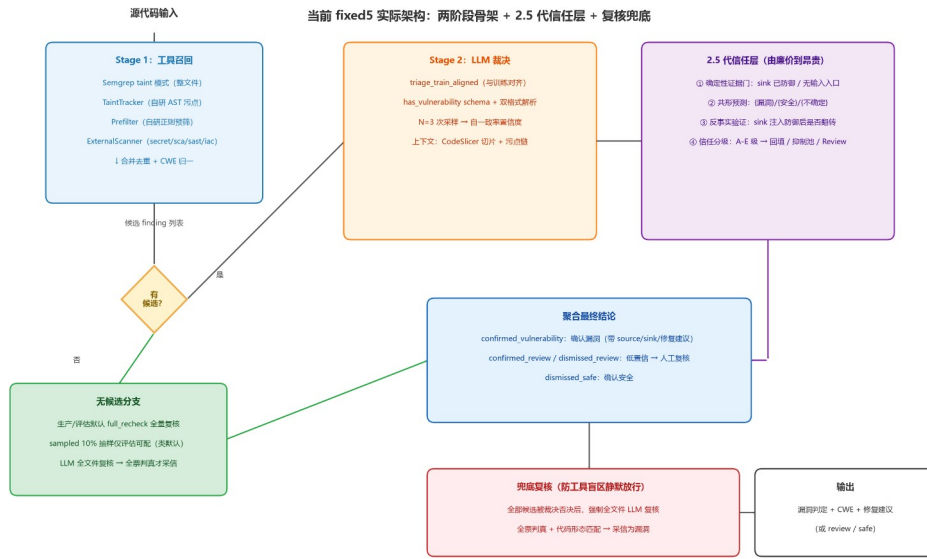


图 3-1 两阶段扫描架构总览

注：界面文字显示为内核开发期代号 Nivis；软件发布名为「凿凿（ZaoZao）」。本图按生产组态绘制：无候选文件执行全量复核（full_recheck）。

3.2 模块划分

系统代码分为服务层、扫描引擎包、前端、插件与启动器五部分。服务层基于 FastAPI，含六个服务模块：scanner（扫描编排与后端构建）、scheduler（优先级调度）、fetcher（网页抓取与 SSRF 防护）、reporter（报告渲染）、multi_model_scanner（多模型投票）、model_registry（模型注册表与默认值解析）。扫描引擎包 graduation_project 含二十余个自研模块，覆盖污点分析、预筛、切片、裁决、信任层、纠正与导出全链路，核心模块及职责如表 3-1 所示。

表 3-1 扫描引擎核心模块

模块	职责
two_stage_scanner	两阶段主流程：召回去重、N 次采样裁决、证据门、聚合与三类兜底复核
taint_tracker	基于 tree-sitter 的自研轻量污点分析（五语言）
prefilter	正则规则预筛层，产出候选并标注可信度
code_slicer	长文件 AST 切片（ ≥ 150 行），拼装行号前缀上下文

模块	职责
external_scanner	七种第三方安全工具的统一封装层（SAST/taint/secrets/SCA/IaC）
conformal / counterfactual	共形预测统计门控 / 反事实验证因果门控
cwe_normalizer / line_normalizer	CWE 编号查表纠正 / 行号内容锚定纠正（零 token 开销）
fix_verifier	修复建议语法校验与幻觉行号检测
sarif_report	SARIF 2.1.0 标准化导出
chroma_manager	本地向量知识库管理（强制离线）
prompts / llm_client 等	Prompt 契约与多后端模型客户端抽象

前端为五页面多页应用（欢迎页、仪表盘、扫描工作台、CWE 样本库、安全态势），无构建步骤、无第三方图表库；插件为 VS Code 与 IntelliJ 两端；启动器负责跨平台依赖安装、后端选择与模型迁移。

3.3 扫描引擎主线

扫描主线分四步。第一步 Stage 1 并行召回：Semgrep 污点分析、自研污点分析器、规则预筛、外部工具封装层四路并行（线程池四个 worker，子进程型工具释放 GIL，总延迟从顺序求和降为取最大值），产出带 Source/Sink 证据的候选集，经跨工具去重归并。第二步 Stage 2 封闭裁决：对每个候选构造裁决提示词，只取包含污点行的代码切片作为上下文，模型独立采样 N=3 次投票，以自一致率作为置信度。第三步信任层门控：确定性证据门、共形预测、反事实验证依次校验，低置信判定转人工复核。第四步聚合与兜底：文件级多数票聚合，判安全的文件再经兜底复核出口防工具盲区。

该架构的关键反转在于：大语言模型的任务从「在全文中发现漏洞」的开放生成，变为「对具体候选判定真伪」的封闭判别。每条结论都有工具证据链作为锚点，模型的幻觉空间被压缩到二分类判别之内。

3.4 接口设计

服务层提供 29 个 REST 端点，按功能分组如表 3-2 所示。批量扫描与模型下载采用 NDJSON 流式推送（每行一个 JSON 事件），相比 WebSocket 更简单且兼容性更好；下载流带心跳保活与禁缓冲响应头，防止长连接被代理掐断。

表 3-2 REST 接口分组（共 29 个端点）

分组	主要端点与说明
扫描分析	POST /api/analyze（单文件）、/api/analyze/two-stage（两阶段）、/api/batch（批量，NDJSON 流式）、/api/github-scan（仓库）、/api/fetch-url（网页）
结果与报告	GET /api/report/single、批量报告下载；GET /api/results/{id}（扫描进度与结果查询）
模型管理	GET /api/models（列表与后端形态报告）、POST /api/models/pull、DELETE /api/models/{name}、POST /api/models/switch 等
多模型投票	POST /api/multi-model-scan（多模型分别扫描后聚合投票）
任务队列	GET /api/queue（队列状态）、POST /api/queue/{id}/cancel（取消）等调度接口
健康检查	GET /api/health/live（轻量探针，启动器用）、GET /api/health（深度检查，含模型与外部工具状态）
静态资源	五页面静态文件与健康指示接口

接口语义有两条显性约定：输入超限返回 413（请求实体超限，前端同值先行拦截）；进程内后端不具备运行时拉取/删除模型能力时，模型管理端点返回 501 并附能力清单，前端据此优雅降级。SARIF 导出将置信度映射为标准 level（ ≥ 0.8 为 error、0.5~0.8 为 warning、其余为 note），每条结果携带稳定指纹，可跨扫描去重，与 GitHub Code Scanning、VS Code Problems、JetBrains Qodana 原生互通。

3.5 部署形态与运行环境

模型提供两档部署形态。发布形态：Ollama 后端运行量化发布模型 v9max（GGUF Q4_K_M，约 4.7GB），已发布至模型仓库、可在线拉取，兼容性最好，是一键启动的默认形态。高精度形态：Transformers 进程内后端叠加 LoRA 适配器（Nivis- $\alpha 0.5$ ），以 4-bit 量化基座 + FP16 适配器推理，检出质量更高。后端解析顺序为：环境变量显式设置 > 检测到本地适配器且运行时兼容则选 Transformers > 否则回退 Ollama；检测到适配器但本机硬件不适合时自动回退并打印告警。四种推理

后端（Ollama/Transformers/LlamaCPP/vLLM）跨三平台的支持矩阵如图 3-2 所示：
Ollama 全平台兼容作为兜底，Linux 支持性最好，Windows 端限制较多。

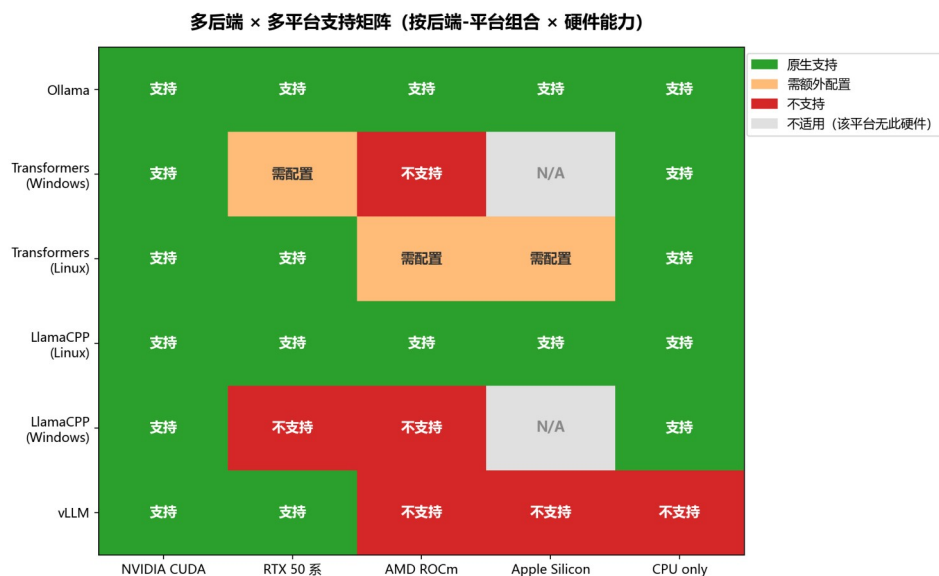


图 3-2 推理后端 × 平台支持矩阵

3.6 效果指标

系统采用统一打分口径：acc 为已裁决样本的准确率；strict 为判真且 CWE 归因正确（含父子族匹配）的严格口径；micro/加权为实例级召回与风险加权分；「未决率」为转入人工复核的比例，与准确率成对引用。两阶段系统在三个测试面上的主要指标如表 3-3 所示。

表 3-3 系统级主要指标（统一打分口径）

系统组态 (测试面)	recall	FPR	acc	未决率	strict	加权
完整信任层 (87 段合成集)	1.000	0.0435	0.987	13.8%	0.923	0.862
纯 LLM 裸 判对照 (87 段合成集)	0.967	0.154	0.931	0	0.898	0.829
真实 CVE 修复片段 (20 段)	0.941	—	0.941	15.0%	1.000	0.864

系统组态 (测试面)	recall	FPR	acc	未决率	strict	加权
真实完整文件首跑（50段）	0.645	—	0.645	38.0%	0.450	0.480

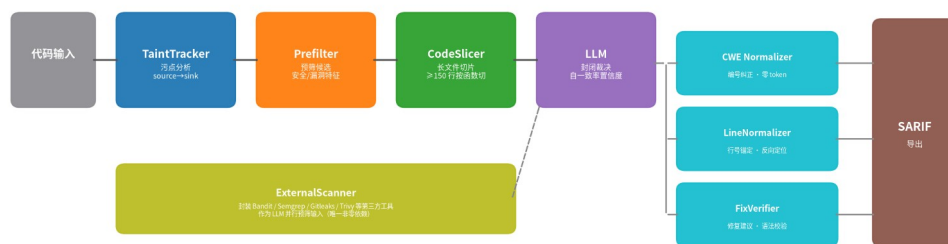
注：87 段合成集上完整信任层较裸判对照 FPR 由 0.154 收敛至 0.0435，归因 strict 由 0.898 升至 0.923——工具证据链反哺归因。真实完整文件面（50 段全为漏洞样本，无安全样本故 FPR 不可计算）两阶段首跑较同面裸判基线 recall +17.8pt、strict +33.5pt，代价是 38% 样本转人工复核，系统在真实文件上偏保守；漏报拆解显示召回缺口主要在 Stage 1 工具召回而非裁决端。发布形态（Ollama Q4 量化）下合成集 recall 0.951 / FPR 0.077；高精度形态（Transformers + LoRA）对应表中数字，界面如实显示当前形态。

4 详细设计说明书

4.1 Stage 1 工具召回设计

Stage 1 由四路并行召回：Semgrep 污点模式、自研污点分析器 TaintTracker、正则预筛 Prefilter、外部工具封装层 ExternalScanner（统一 SAST/污点/密钥/组件成分/基础设施即代码五类扫描，封装 Bandit、Semgrep、Gitleaks、detect-secrets、pip-audit、Trivy 等七种工具）。系统从代码输入到结果导出的自研工具链共八件，其中七件零依赖自研，如图 4-1 所示。

自研工具链：从代码输入到 SARIF 导出的完整流水线



8 个工具：7 个零依赖自研（tree-sitter / 标准库 / 手写状态机）+ ExternalScanner（第三方工具封装层）

图 4-1 自研工具链全景

TaintTracker（自研污点分析）。基于 tree-sitter 语法树做线性数据流分析：外部输入赋值进入污染集，污染变量到达危险函数参数即报路径，不再做来源×危险点的笛卡尔积。三项关键设计：一是语境安全判定——subprocess 列表参数、模板 kwargs 传参、白名单前缀校验链等「字面命中但语境安全」的场景在工具层直接消

除；二是 Web 框架参数自动识别为污染源（路由装饰器参数、@RequestParam 等注解参数）；三是按危险度排序截断，每个作用域最多输出 50 条路径，高危优先，避免低危候选挤占裁决注意力。

Prefilter（规则预筛）。在调用模型前以正则规则预筛：命中安全模式且未命中漏洞特征可判安全，仅命中漏洞特征判可疑，两者都命中或都未命中交由模型复核。全部规则统一关联类型/CWE/风险元数据（单一事实来源）；超过 150 行的长文件跳过安全规则只跑漏洞规则，防止「前半段参数化掩盖末尾漏洞」；硬编码凭证识别用负向后行断言修正下划线前缀漏配。独立评估显示：预筛在被调用子集上的严格归因准确率 0.917。

切片与上下文对位。文件不足 150 行整体送审；超过则按顶层函数/类方法切片，每片带文件级上下文头与类骨架。切片拼装保证行号精确对位：头部不带行号，函数体逐行加「N|」前缀，模型输出的行号可与源码直接对位。自研的 Semgrep 污点规则四件套（SQL 注入/命令注入/代码注入/XSS）则对整文件做污点扫描，避免切片把同一条数据流的首尾割裂。

四路候选经跨工具去重归并后进入 Stage 2：同一数据流只裁决一次，无证据候选归并到已有证据的同条流，无法归并的退化为「类型+规则」键避免误合。

4.2 Stage 2 封闭裁决设计

裁决只回答一个封闭问题：「这条工具告警是否为真实漏洞」。每个候选独立采样 $N=3$ 次，置信度取自一致率（有效票占比）——解析失败或推理出错的票计为无效票，不参与分母，避免把「模型失效」稀释成「低置信否决」；全部票无效时置信度归零、转人工复核。解析失败时以 JSON 约束解码重试一次，数学上保证输出可解析。

每次裁决按「是否确认 + 置信度」落入四个档位：高置信确认（ ≥ 0.8 ）、低置信确认（ $0.5 \sim 0.8$ ）、高置信否决（ ≥ 0.8 ）、低置信否决（ $0.5 \sim 0.8$ ）；后三档均进入人工复核清单，仅高置信确认直接计为漏洞。文件级聚合采用多数票：任一候选多数票判真即文件判真；漏洞类型投票时区分「独立票」与「回声票」——模型输出类型与工具标注相同只是复读，独立判断出的类型在平票时胜出。

裁决格式与模型训练格式严格对齐（字段级一致），避免「训练时见过的格式、推理时换成另一种」导致的自一致漂移。针对模型对工具锚点的过度顺从，提示词按来源注入信任标注：低信任的位置型规则告警显式提示「此告警无数据流证据链、

历史误报率高，须独立分析」，污点类标注「可信度较高」，预筛类标注「可信度中等」。

4.3 2.5 代信任层

裁决结论并不直接采信，而是经过四层门控（图 4-2）：确定性证据门、共形预测、反事实验证与信任分级回填。设计动机来自一个实测发现：在高精度后端上模型会「全票但错」——三次采样全票确认为误报的候选，依赖投票分歧的统计门控在过度自信的后端上结构性失明，因此必须有与后端无关的确定性校验兜底。

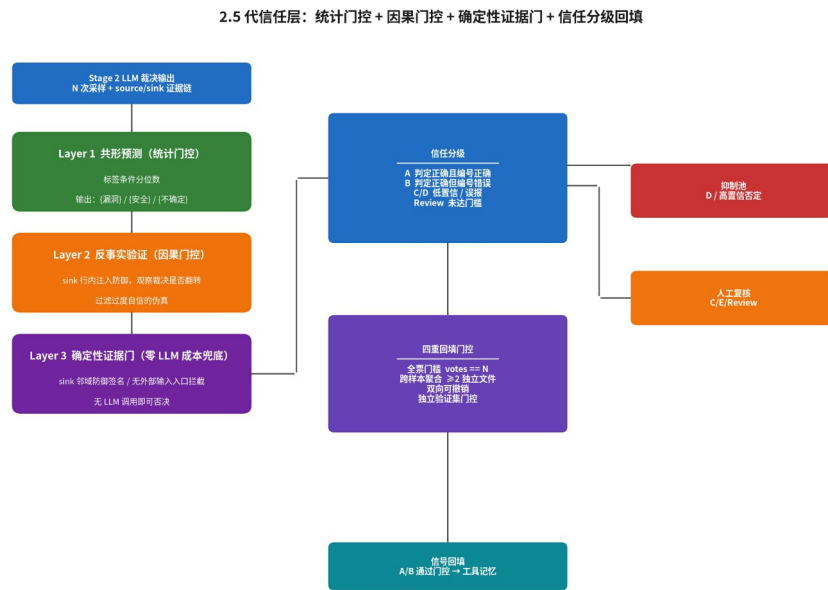


图 4-2 2.5 代信任层结构

确定性证据门（零模型成本）。两条与后端无关的静态校验：其一，危险点邻域（前 4 后 3 行）已含该漏洞类型的已知防御特征（参数化查询、shlex.quote、autoescape 等）而模型仍判漏洞，视为疑似模式匹配误报；其二，纯字面量脚本且无任何外部输入入口时污点无从产生。命中者不直接否决，而是排除出「直接判漏洞」的依据转人工复核——门是保守的，宁可复核不错杀。

共形预测（统计门控）。以标签条件非一致性分数做三类预测：{漏洞}、{安全}、{不确定}。校准集拟合分位数阈值（默认覆盖率 90%），阈值随模型导出持久化；校准文件缺失时安全降级为未校准模式（全部走不确定档、回退投票逻辑），降级可审计而非异常。只有落入单元预测集的判定才有资格参与后续工具回填，「弃权」被数学化地排除在外。

反事实验证（因果门控）。对判漏洞的候选，在危险点行内注入最小防御改写（拼接改参数化、加消毒包裹、弱反序列化改安全替代），保持语义不变、只动防御，再让模型重判：若结论翻转为安全，说明模型理解的是「防御的存在性」而非表面模式；若不翻转且原代码已含同类防御，则该候选降级复核。扰动只限危险点行内、裁决只用单次低温采样，成本可控。

信任分级回填（模型反哺工具）。模型输出按判定质量分 A~E 五级，回填结构如图 4-3：仅高置信判定经四重门控（全票门槛、跨独立文件聚合、双向可撤销、独立验证集复核）后写回信号注册表——高置信误报所涉规则进入抑制池，高置信确认的规则获得候选排序加权；单次否定不抑制、两次高置信假阴性可解锁，防止模型错误污染确定性工具。确定性工具（密钥/组件扫描）自判结果不受模型意见影响，保持分层免疫。

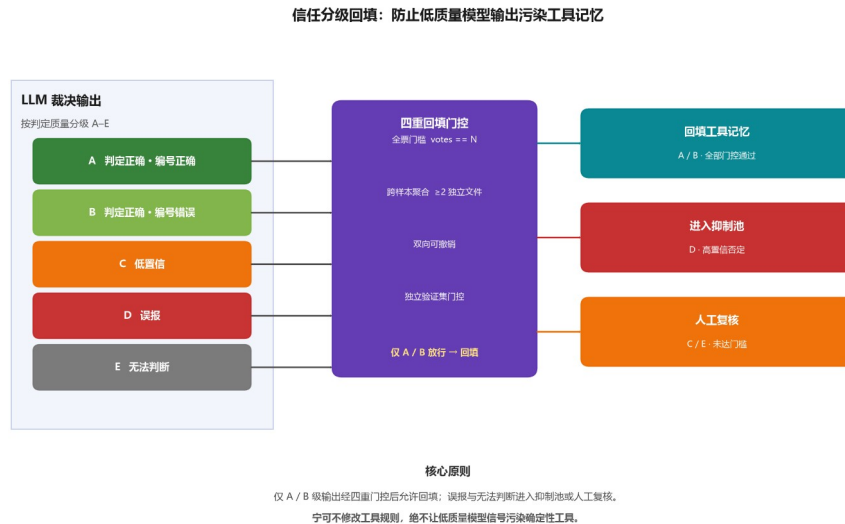


图 4-3 信任分级回填与抑制池

4.4 复核兜底三出口

裁决式任务只判断「工具告警是否为真」，模型不会自主发现工具没召回的漏洞；规则覆盖不可能百分之百，工具盲区会在裁决路径上静默放行。为此系统设置三个复核出口：其一，无候选文件全量复核（生产默认组态）：工具零召回的文件整体送模型开放式分析，消除「无证据判安全」的静默放行；其二，无候选抽样复核（评估可选组态）：按 10% 比例抽样复核，复核票数上限 3、温度取高值打破同模态重复；其三，裁决全否决兜底：全部候选被否决的文件再做一次全文件复核。三个出口的复核结果均需全票判真才采信，且采信前经形态校验：注入型漏洞必须

存在标准危险点特征且邻域无标准防御；认证缺失、CSRF 等「缺失型」漏洞无确定性验证手段，全票采信后如实标注「模型语义兜底」。模块级计数器在线估计工具层漏报率，把「工具召回漂移」变成可观测指标。

4.5 关键工程机制

优先级调度。基于堆的优先级队列：交互式扫描排高优先级、批量扫描排低优先级，同优先级先到先排；每个客户端最多排队 8 个任务防止单批霸占；工作线程把结果回填到异步事件循环，取消任务直接移出队列释放配额。插件请求头声明扫描范围，后端据此分派优先级。

CWE 编号与行号自动纠正。两项查表式后处理均不进入模型上下文、零 token 开销。CWE 纠正处理「语义对、编号错」的记忆性错误：关键词按具体度排序防子串误归一，证据守卫四级优先避免破坏性覆盖，父子族双向遍历兼容父类/子类口径。行号纠正利用「模型数错行但行内容通常可靠」的规律：以行内容为锚在源文件中定位真实行号，三级评分、高分优先、同分取距原行号最近者，匹配不到可靠锚点则原样保留。

修复建议行号锚定契约。修复建议统一为「行号锚定的单行局部改法」而非完整代码块，配套校验器抓幻觉行号：Java 片段双候选包壳编译校验，JavaScript 用语法检查并兼容 ES 模块；危险模式表覆盖命令执行、弱哈希、开放重定向等 12 类补充模式，规则来源为评估中失败案例的根因分析。

长文件与批量解码。切片后聚合语义保守：任一片段判漏洞即整文件判漏洞，取风险最高片段的详情；支持批量解码的后端一次前向处理完整批次（必须左填充，短序列批量解码否则不可靠），批量失败逐条回退而非整批报废；耗时统计从整文件分析起点计时，长文件与排队耗时如实呈现。

安全防护与下载工程。网页抓取实现多层 SSRF 防护：协议白名单、内网地址拦截、DNS 重绑定缓解（双重解析一致性校验）、逐跳重定向复检（最多 5 跳）、响应体 2MB 截断。国内下载四件套环环相扣：镜像端点在任何依赖库导入前设置、镜像域名绕过代理直连、代理协议规范化、断点续传 + 指数退避 + 完整性校验。

4.6 设计依据与选型论证

判别优于生成：把模型任务锁定为对具体候选的二分类判别，是整个架构的第一性选择——开放生成任务里模型需要同时完成「发现」与「判定」，幻觉无法锚

定；封闭判别让每条结论都挂在工具证据上，错误可归因到「工具没召回」或「裁决错判」之一，为归因分流提供结构性基础。

自一致率替代自报置信度：大模型自报的置信度校准性差，而 N 次采样的自一致率是多数表决的软化，统计性质清楚、无需额外训练即可用作置信度代理。共形预测与反事实验证的组合来自技术调研：统计门控成本低但有「过度自信后端失明」的结构盲区，因果门控与后端无关、成本略高，二者互补；确定性证据门则以零模型成本拦截两类高频误报模式。

工具-模型自适应闭环替代「按类型补规则」：实测中针对误报逐类补规则是死路——规则间相互作用使误报按下一样本漂移。替代方案是让工具做确定性召回、模型做语义裁决、裁决置信经门控后回填工具记忆（抑制池与排序加权），工具与模型互为喂养、共同进化，且回填门槛参数化写入代码，防止模型的错误判定污染确定性工具。

5 数据存储设计

5.1 总体说明

本系统无独立数据库服务，全部状态采用四类本地持久化：配置与信号注册表等 JSON 文件、模型权重与规则文件、本地向量知识库、浏览器端 `localStorage`。这一设计服务于两个目标：完全离线可用（无数据库连接依赖），以及绿色可卸载（全部数据限定在项目目录内，卸载脚本分级清理）。

5.2 文件存储

表 5-1 本地文件存储布局（项目目录内）

路径	内容
<code>models/signal_registry.json</code>	信号注册表：工具规则置信状态、抑制池、回填确认记录（原子写入：临时文件 + 原子替换）
<code>models/conformal_calibration.json</code>	共形预测校准阈值（评估侧拟合、生产端加载，可审计降级）
<code>models/semgrep_rules/</code>	本地化 Semgrep 规则包（离线可用，无需在线拉取规则）

路径	内容
models/adapter_*/	LoRA 适配器权重（含训练配置），高精度形态按目录探测自动启用
models/ollama/	发布形态量化模型（启动器锁定模型存储到项目目录，不占用系统盘）
models/transformers/	本地基座模型（高精度形态使用）

5.3 向量知识库

漏洞知识库采用嵌入式向量数据库 Chroma，持久化于 data/chroma_db，集合名 vuln_knowledge。向量化模型选用多语言模型 BAAI/bge-m3（知识库以中文漏洞资料为主，纯英文小模型中文检索质量不足）；运行时强制本地离线模式，绝不联网下载。检索参数：以源码前 2000 字符为查询、返回前 3 条；知识库初始化失败时静默降级为不检索并记忆失败状态，避免反复重试——知识库增强属可选开关，开启后推理时间增加不足 2%。

5.4 前端持久化

浏览器端以 localStorage 持久化主题设置、扫描轮次、扫描结果与报告历史，旧结构自动迁移；批量扫描以双保险机制保证「一轮只入账一次」。深度健康检查结果以 sessionStorage 短期缓存，页面切换时主动中止进行检查请求，避免占满同域并发连接。

5.5 数据生命周期

运行期临时数据：仓库扫描浅克隆至系统临时目录，120 秒超时、禁用交互凭据提示，扫描后立即递归删除；无任何代码内容落盘到项目目录。卸载：卸载器支持试运行（只列将删内容）、保留 Ollama 与模型、仅清依赖不删项目三种分级；外部安全工具为系统级安装，不随卸载自动删除并在卸载时提示。合规边界：运行期推理框架强制离线模式，除用户主动的首次下载外不产生对外网络请求。

6 软件界面设计书

6.1 设计系统

界面遵循「专业的安全工具应该是隐形的，数据大于装饰」的设计哲学：不以动画堆砌吸引眼球，而把数据清晰度放在首位。实现上采用双层设计令牌体系：底层 `design-tokens.css` 以 CSS 变量定义颜色、圆角、间距、阴影与字体，经语义映射为统一的界面令牌，五个页面共用；动效层统一定义缓动曲线、玻璃卡片与导航样式。深浅双色主题完整覆盖；页面头部内联脚本在样式表加载前读取主题设置，规避刷新闪烁；尊重系统的减弱动态效果偏好。

6.2 信息架构与页面清单

表 6-1 页面清单

页面	职责
欢迎页	首次访问引导与入口导航
仪表盘	历史扫描统计、引擎健康状态、最近发现的漏洞
扫描工作台	核心功能页：粘贴/上传/批量/GitHub/网页五类扫描入口与结果区
CWE 样本库	16 类常见漏洞的可搜索示例与修复方案，支持深链接直达
安全态势	整体安全态势总览与趋势

页面间以深链接互通（样本直达、标签定位、文件定位三类参数），形成从「扫描结果 → CWE 样本解释 → 发起下一次扫描」的闭环。

6.3 界面总览

主要界面如图 6-1 至图 6-6 所示（界面文字为内核开发期代号 Nivis，软件发布名为「凿凿（ZaoZao）」）。

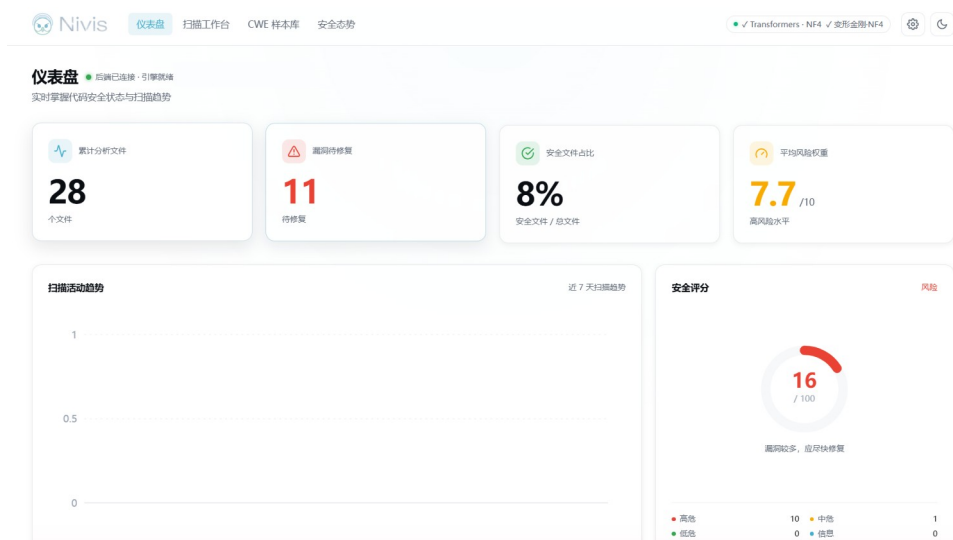


图 6-1 仪表盘：扫描统计与健康状态

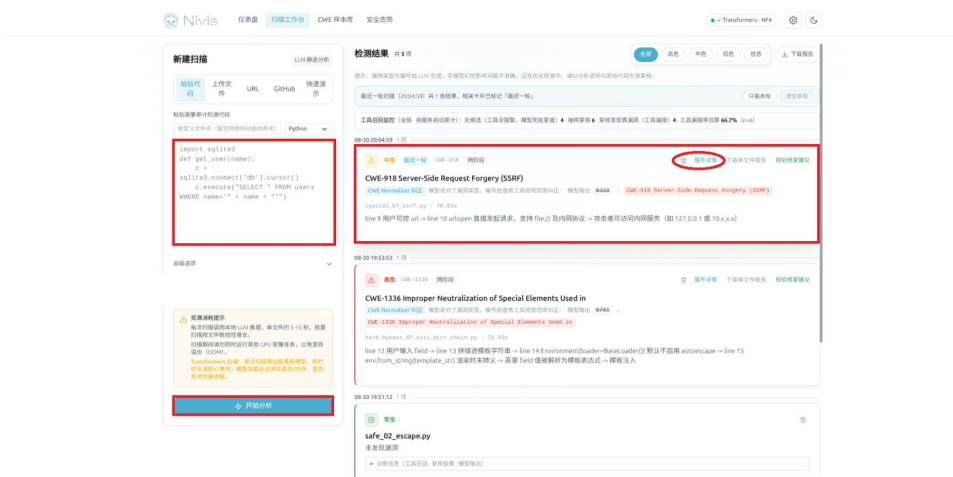


图 6-2 扫描工作台：五类扫描入口与结果区



图 6-3 结果详情：CWE 归因、证据链与修复建议

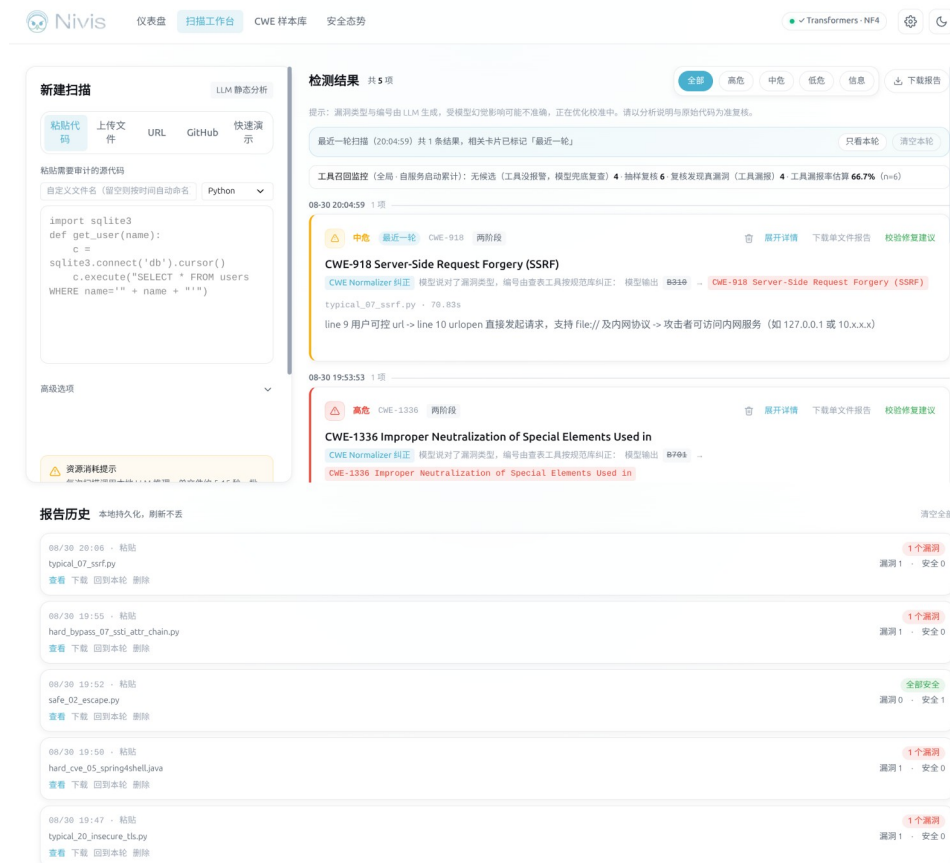


图 6-4 报告历史：Markdown 报告与 SARIF 导出

Nivis

仪表盘 扫描工作台 CWE 样本库 安全态势

Transformers · NF4 变形金刚 NF4

CWE 样本库

浏览常见安全弱点分类、严重程度及代码示例，辅助审计与学习。
漏洞原理参考库：如需扫描真实代码，请前往 扫描工作台 · 快速演示

搜索 CWE ID、名称或关键词...

全部 高危 中危 低危 信息

共 19 个 CWE · 显示 19 个

最近扫描中发现 5 类 CWE

参考库 · 实时推送 · 点击卡片查看代码示例

CWE-79 高危 近期发现 5 次

跨站脚本攻击 (XSS)

未对用户可控输入进行充分校验或转义，导致攻击者可在受害者的浏览器中注入并执行恶意脚本，窃取 Cookie 或劫持会话。

Web 网络 JavaScript PHP

查看详情 → 快速演示 (3)

CWE-89 高危 近期发现 1 次

SQL 注入

应用程序将不可信数据直接拼接到 SQL 查询中，使攻击者能够篡改查询语句，读取或修改数据库内容，甚至执行系统命令。

Java PHP Python

查看详情 → 快速演示 (6)

CWE-78 高危

命令注入

程序将不可信数据拼接到操作系统命令中执行，攻击者可通过命令分隔符或元字符注入并执行任意系统命令。

Python Shell 壳牌 Go 去吧

查看详情 → 快速演示 (5)

CWE-22 高危 近期发现 2 次

路径遍历

程序使用外部输入构造路径时未做限制，攻击者可利用 / 等符号访问目录之外的文件或资源，导致敏感信息泄露。

Java Go 去吧 Node.js

查看详情 → 快速演示 (6)

CWE-94 高危

代码注入 (eval/exec)

应用程序将不可信数据作为代码执行 (如 eval, exec)，攻击者可注入并运行任意代码，造成服务器沦陷。

Python PHP

查看详情 → 快速演示 (2)

CWE-502 高危

不安全的反序列化

程序仅序列化来自不可信来源的数据，攻击者可构造特定对象触发任意代码执行或拒绝服务。

Java Python .NET

查看详情 → 快速演示 (4)

CWE-1136 高危

服务端模板注入 (SSTI)

应用将用户输入拼接到模板表达式后由模板引擎求值，攻击者可注入模板语法实现远程代码执行。

Python Java

查看详情 → 快速演示 (3)

CWE-611 高危

XML 外部实体 (XXE)

XML 解析器处理外部实体声明，攻击者可通过构造恶意 XML 读取服务器文件、发起 SSRF 或造成拒绝服务。

Java Python

查看详情 → 快速演示

图 6-5 CWE 样本库：可搜索示例与修复方案

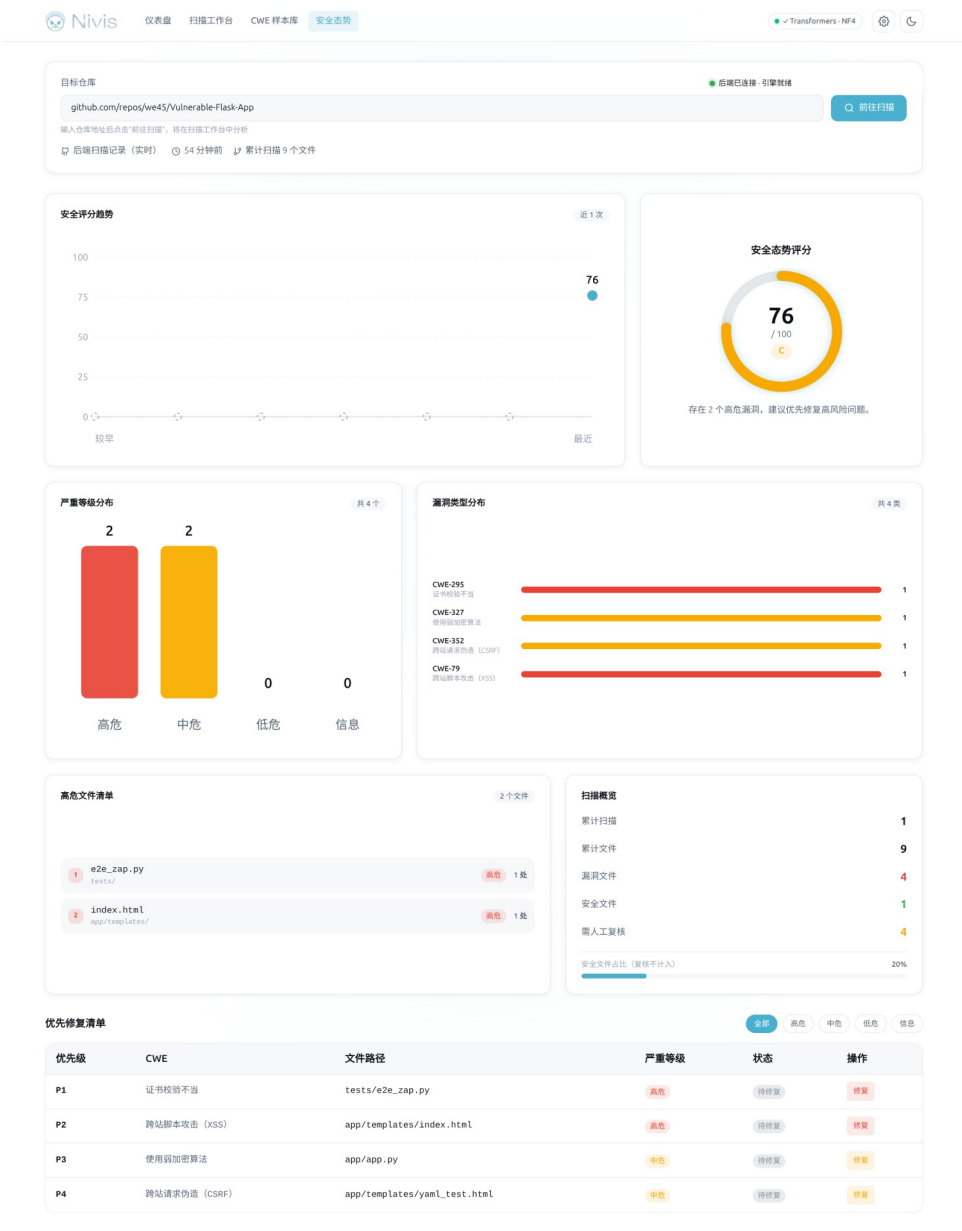


图 6-6 安全态势总览

编辑器插件界面包含 VS Code 诊断波浪线与信任层明细面板、IntelliJ 气球通知两种形态。【截图待补：VS Code 插件行级标记与结果面板；IntelliJ 插件气球通知】

6.4 关键交互

评分圆环以三角函数逐帧重算圆弧路径，从根源规避浏览器圆头线帽与虚线动画叠加产生的渲染位移；全部图表（趋势折线、环形分布、柱状图、评分圆环）为零依赖手写 SVG，离线可用。主题切换采用视图过渡 API 实现交叉淡化，带防重入

锁并跨标签页同步，不支持的浏览器直接切换。批量扫描以 NDJSON 流式逐行解析进度，扫完一个即入账一个。深度健康检查最坏耗时 30 秒以上，以会话级缓存避免重复请求。

7 用户操作手册

7.1 三个最常用的任务

任务 A（扫一段代码/一个文件）：打开本机服务地址进入扫描工作台；在「粘贴代码」标签选择语言、粘贴代码后点「开始分析」，或在「上传文件」标签拖入文件；等待十几秒后结果区给出判定、CWE 类型、风险等级、原理说明与修复建议。

任务 B（批量扫描并导出报告）：在工作台「上传文件」一次拖入整个模块的代码文件（最多 200 个）；点「开始分析」后页面实时显示进度；完成后点「下载报告」得到 Markdown 报告（漏洞汇总表 + 逐文件结果 + 修复建议），亦可导出 SARIF。

任务 C（扫 GitHub 仓库/网页）：「GitHub」标签粘贴仓库地址、设置最多扫描文件数后开始，系统浅克隆、扫完即删；「URL」标签输入网页地址即可分析页面脚本。

7.2 读懂扫描结果

判定语义、结果字段与安全评分见 2.3 节。使用中需注意：「需人工复核」不是出错而是设计；扫描结果与 Bandit/Semgrep 存在三类正常差异（放行有效防御、报出语义类漏洞、低置信转复核）。

7.3 Web 界面导览

五页面职责见 6.2 节表 6-1。界面右上角为模型管理抽屉；所有页面支持深浅色主题切换，图表离线可用。

7.4 在编辑器里使用

前置条件：本机后端服务已启动。VS Code：在扩展面板选择「从 VSIX 安装」并选择作品附带的插件包；打开代码文件后右键「分析当前文件」，结果面板展示判定、修复建议与信任层明细，确认漏洞按行号标红波浪线；命令面板可批量扫描

工作区；常用设置包括后端地址、保存自动扫描（1.5 秒防抖）与批量文件上限。

IntelliJ IDEA：从磁盘安装作品附带的插件包并重启；选中代码（不选则扫整个文件）右键扫描或用快捷键，结果以气球通知展示。

7.5 命令行使用

命令行无需先启动后端服务，直接复用扫描引擎：

```
python -m app.launcher.vuln_scanner_cli <命令>
health          # 健康检查（模型/服务是否可用）
scan app/main.py # 扫单个文件（--format json 可输出 JSON）
batch ./src --output report.md # 批量扫目录并导出报告
url https://example.com      # 扫网页脚本
github <仓库地址>           # 扫 GitHub 仓库（浅克隆）
```

通用参数：--model 切换模型、--rag 开启知识库、--format 选择输出格式。扫描失败时退出码为 1，可直接接入 CI 做提交前检查。

7.6 离线使用与模型管理

联网仅在首次下载依赖与模型时需要，运行期完全离线。模型管理优先使用 Web 界面的模型抽屉（拉取/删除/切换，带实时进度），亦可用命令行参数或环境变量切换。两档精度形态见 3.5 节，界面如实显示当前形态；国内网络环境一般无需手动配置代理。

7.7 常见问题与处理

表 7-1 常见问题与处理

现象	处理
扫描很慢（超过 60 秒/文件）	大概率在用 CPU 推理：查看启动日志硬件检测段；有独立显卡却走 CPU 时重启启动脚本，或显式设置后端环境变量为 ollama
结果全是「需人工复核」	模型可能未正确加载：运行 health 命令检查，确认模型名一致
显存不足（OOM）	调小上下文窗口：设置环境变量 VULN_SCANNER_NUM_CTX=4096
启动报端口被占用	查找并结束占用 8765 端口的进程后重启
VS Code 插件没反应	确认后端已启动且健康接口可访问；核对插件后端地址设置

现象	处理
红波浪线没显示	插件设置 <code>markDiagnostics</code> 需为开启；个别无法定位到行的情况只在面板展示
Linux 上模型没进项目目录	系统自带 <code>ollama</code> 服务抢占：停用系统服务后重启启动脚本

8 安装手册

8.1 环境要求

表 8-1 硬件与系统要求

项目	最低要求	建议配置
操作系统	Windows 10+ / Linux / macOS	—
Python	3.10 及以上（启动脚本自动检测）	—
磁盘	8GB（模型约 5GB + 依赖约 2GB）	15GB
内存	8GB	16GB
显卡	可无（自动回退 CPU，慢 10~20 倍）	NVIDIA/AMD/Apple 独显，显存 $\geq$ 4GB

8.2 获取与一键启动

作品源码随提交材料以 ZIP 包提供，解压后进入项目目录：Windows 双击 `app\launcher\start_windows.bat`；Linux/macOS 执行 `bash app/launcher/start_linux_macos.sh`。启动器全自动完成后续步骤，用户只需等待。

8.3 首次启动流程

首次启动依次自动完成五步：（1）安装 Python 依赖（国内自动走清华镜像）；（2）自动选择推理后端：检测到作品自带模型且显卡支持则用高精度形态，否则用发布形态（首次自动安装 Ollama 运行时）；（3）下载扫描模型（约 5GB）；（4）安装传统安全工具（缺哪个跳过哪个，不影响运行）；（5）启动服务并自动打开浏览器。首次启动耗时取决于网速（主要是模型下载）；之后每次启动通常 10 秒内完成。模型存储锁定在项目目录，不会占用系统盘。

8.4 确认安装成功

浏览器打开 `http://localhost:8765`，看到欢迎页或仪表盘即成功；亦可在终端运行健康检查命令验证模型与服务状态。

8.5 卸载

表 8-2 卸载方式

平台	操作
Windows	双击 <code>uninstall_windows.bat</code> （或运行 <code>python uninstall.py</code> ）
Linux / macOS	执行 <code>bash uninstall.sh</code> （或 <code>python3 uninstall.py</code> ）

常用参数：`--dry-run` 只列出将删内容；`--keep-ollama` 保留 Ollama 与模型；`--keep-project` 只清依赖不删项目目录。须用安装时的同一个 Python 环境运行卸载（conda/venv 用户先激活原环境）；外部安全工具不会被自动删除，需要时手动卸载。

附录

附录 A 命令行速查

表 A-1 命令行速查

命令	作用	常用参数
<code>health</code>	健康检查（模型/服务/外部工具）	—
<code>scan <文件></code>	扫描单个文件	<code>--format json</code> ； <code>--model</code> ； <code>--rag</code>
<code>batch <目录></code>	批量扫描目录并导出报告	<code>--output</code> ； <code>--limit</code> ； <code>--no-recursive</code> ； <code>--max-files</code>
<code>url <网址></code>	扫描网页内脚本	<code>--format</code>
<code>github <仓库></code>	浅克隆扫描 GitHub 仓库	<code>--max-files</code> （默认 50 上限）

附录 B 输入限制与支持范围

表 B-1 输入限制与支持范围

项目	限制
代码语言	Python / JavaScript / TypeScript / Java / PHP / Go / HTML
文件类型	.py .js .ts .jsx .tsx .java .php .go .html .htm .vue .svelte
单段代码	最多 200 万字符
单文件	最大 2MB
单次批量	最多 200 个文件、总计 10MB
仓库扫描	浅克隆 + 120 秒超时 + 文件数上限（默认 20，硬上限 50）

附录 C 排查速查

常见现象与处理见 7.7 节表；补充三条：**RAG** 相关报错多为知识库未初始化或旧向量库需重建；模型下载中断后重启脚本自动断点续传；两档部署形态精度不同属正常现象，界面会显示当前形态。